



Cyberscope

A *TAC Security* Company

Audit Report

Stipent

September 2026

Network : POL

Address: 0x2Af618611967Ddfb05bC94d64cb6F4B523E10D36

Audited by © cyberscope

Table of Contents

Table of Contents	1
Analysis	2
Diagnostics	3
Risk Classification	4
Review	5
Audit Updates	5
Findings Breakdown	6
ST - Stops Transactions	7
Description	7
Recommendation	7
Team Update	8
PFVOR - Permanent Freeze Via Ownership Renouncement	9
Description	9
Recommendation	10
Team Update	10
L15 - Local Scope Variable Shadowing	11
Description	11
Recommendation	11
Team Update	11
L19 - Stable Compiler Version	12
Description	12
Recommendation	12
Team Update	12
Functions Analysis	13
Inheritance Graph	14
Flow Graph	15
Summary	16
Disclaimer	17
About Cyberscope	18

Analysis

Severity	Code	Description	Status
●	ST	Stops Transactions	Multisign
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

Severity	Code	Description	Status
●	PFVOR	Permanent Freeze Via Ownership Renouncement	Acknowledged
●	L15	Local Scope Variable Shadowing	Acknowledged
●	L19	Stable Compiler Version	Acknowledged

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	Stipent
Optimization	200 runs
Explorer	https://polygonscan.com/address/0x2af618611967ddfb05bc94d64cb6f4b523e10d36
Address	0x2Af618611967Ddfb05bC94d64cb6F4B523E10D36
Network	POL
Symbol	STPN
Decimals	18
Total Supply	10,000,000,000

Audit Updates

Initial Audit	28 Sept 2026
----------------------	--------------

Findings Breakdown



● Critical	1
● Medium	0
● Minor / Informative	3

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	1
● Medium	0	0	0	0
● Minor / Informative	0	3	0	0

ST - Stops Transactions

Criticality	Critical
Location	contracts/contract-5aa9b74b77.sol#L36
Status	Multisign

Description

The contract owner has the authority to stop the transactions for all users. The owner may take advantage of it by calling the `pause` function. As a result, the contract may operate as a honeypot. In addition, the transactions may be stopped permanently as described in the PFVOR finding.

```
function pause() public onlyOwner {  
    _pause();  
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership while the contract is unpaused, which will eliminate the threats but it is non-reversible.

Team Update

The team has acknowledged that this is not a security issue and states:

The pause function is kept only as an emergency safeguard, for example in case of a security incident, and has never been used since deployment. Contract ownership has been transferred to a 2-of-3 Safe multisig on Polygon, so no single wallet can pause transfers.

Multisig (new owner)	0xDE4A956D470B2017543d844d36727d0C9B6494Bd
Threshold	2 of 3 signers (Safe v1.5.0, no modules or guard)
Transfer transaction	0x3152e736863a18ebd2d618acc7d29f729685a7353e9f5ca9b475c3aae0e93766
Executed	24 September 2026, 14:06 UTC, block 94369095

PFVOR - Permanent Freeze Via Ownership Renouncement

Criticality	Minor / Informative
Location	contracts/contract-5aa9b74b77.sol#L16,36,40
Status	Acknowledged

Description

The contract does not verify the paused state at the time ownership is renounced. The contract inherits the standard `Ownable` implementation and does not override the `renounceOwnership` function, while the `pause` and `unpause` functions are both restricted to the owner through the `onlyOwner` modifier. If the owner pauses the contract and subsequently renounces ownership, the `unpause` function can never be called again, because no address holds the owner role. In this state the `_update` function reverts for all transfers and burns, and the contract offers no recovery path, so holder balances become permanently non-transferable. The same state is reached when ownership is transferred to an address that cannot act, since `transferOwnership` completes in a single step without any confirmation from the new owner.

```
function pause() public onlyOwner {
    _pause();
}

function unpause() public onlyOwner {
    _unpause();
}

...
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}

function transferOwnership(address newOwner) public virtual onlyOwner {
    if (newOwner == address(0)) {
        revert OwnableInvalidOwner(address(0));
    }
    _transferOwnership(newOwner);
}
```

Recommendation

The team is advised to ensure that the paused state cannot become permanent through the loss of the owner role. The removal of the pause mechanism also resolves this finding.

Team Update

The team has acknowledged that this is not a security issue and states:

The contract has never been paused. Ownership is held by a 2-of-3 multisig, which removes the risk of losing access through a single key, and the team will not renounce ownership while the contract is paused.

L15 - Local Scope Variable Shadowing

Criticality	Minor / Informative
Location	@openzeppelin/contracts@5.0.0/token/ERC20/extensions/ERC20Permit.sol#L39
Status	Acknowledged

Description

Local scope variable shadowing occurs when a local variable with the same name as a variable in an outer scope is declared within a function or code block. When this happens, the local variable "shadows" the outer variable, meaning that it takes precedence over the outer variable within the scope in which it is declared.

```
string memory name
```

Recommendation

It's important to be aware of shadowing when working with local variables, as it can lead to confusion and unintended consequences if not used correctly. It's generally a good idea to choose unique names for local variables to avoid shadowing outer variables and causing confusion.

Team Update

The team has acknowledged that this is not a security issue and states:

This occurs inside the standard OpenZeppelin v5.0.0 library (ERC20Permit) and has no security impact.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	contracts/contract-5aa9b74b77.sol#L2
Status	Acknowledged

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
pragma solidity ^0.8.20;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Team Update

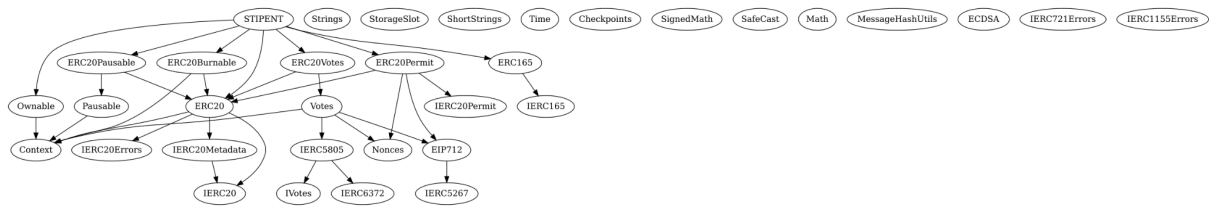
The team has acknowledged that this is not a security issue and states:

The contract is already deployed and was compiled with a single fixed compiler version. We will lock the pragma in any future contract.

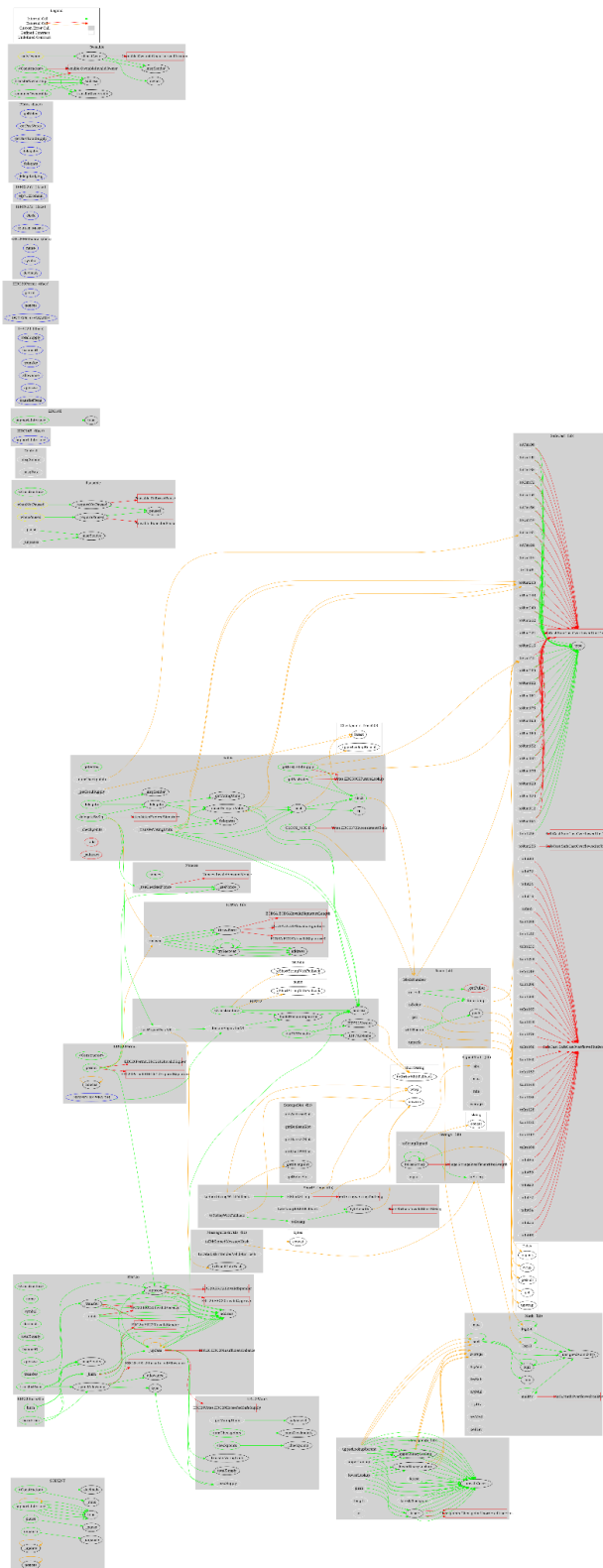
Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
STIPENT	Implementation	ERC20, ERC20Burnable, ERC20Pauseable, Ownable, ERC20Permit , ERC20Votes, ERC165		
		Public	✓	ERC20 Ownable ERC20Permit
	supportsInterface	Public		-
	pause	Public	✓	onlyOwner
	unpause	Public	✓	onlyOwner
	_update	Internal	✓	
	nonces	Public		-

Inheritance Graph



Flow Graph



Summary

Stipent contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the multisig wallet that holds the owner role, like stop transactions. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io